FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery

Part II — Runtime Invariant Algebra

Sizhe Tan
with chatGPT (OpenAI)
2026-07-15

Repository: https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA

Abstract

The Common Concept Core (CCC) was originally introduced as a structural mechanism for identifying the shared semantic core among multiple observations.
Within the Runtime Invariant Algebra proposed by FTRIA, CCC can be reinterpreted at a higher level.
Rather than viewing CCC as an isolated algorithm, this paper interprets CCC as an **explicit Runtime Invariant Discovery Operator**.
Under this interpretation, CCC performs the discovery of stable Runtime Invariants from multiple structural realizations.
This reinterpretation naturally connects CCC with modern representation learning, Transformer embeddings, graph matching, and numerous existing AI approaches, placing them within a common Runtime Invariant Discovery Algebra.

1. Existing Algorithm

The Common Concept Core (CCC) identifies structural elements shared by multiple observations.
Conceptually,
Observation A

Observation B

Observation C

↓

Common Concept Core

The discovered core represents the stable semantic structure common to all observations.
Originally,
CCC was developed as a structural discovery mechanism supporting semantic understanding, retrieval, generation, and structural reasoning.

2. Traditional Interpretation

Traditionally,
CCC has been understood as

- common semantic extraction,
- structural intersection,
- concept discovery,
- shared representation,
- semantic core identification.

Its emphasis has been identifying commonality while suppressing observation-specific variations.
This interpretation successfully explains the practical behavior of CCC, but does not fully reveal its mathematical role.

3. Runtime Invariant Interpretation

Within FTRIA,
CCC may be interpreted as an **explicit Runtime Invariant Discovery Operator**.
Instead of discovering only a semantic core,
CCC discovers a Runtime Invariant.
Conceptually,
Multiple Runtime Realizations

↓

Runtime Invariant Discovery

↓

Common Concept Core

The Common Concept Core therefore represents an explicit structural approximation of the Runtime Invariant shared by multiple realizations.

Under this perspective,
CCC is not merely a semantic operation.
It is a Runtime Invariant Discovery process.

4. Why CCC Naturally Discovers Runtime Invariants

Runtime Invariants remain stable across admissible runtime variations.
CCC follows exactly the same principle.
Given multiple implementations,
representations,
or observations,
CCC suppresses implementation-specific differences while preserving their common structural identity.
Consequently,
CCC performs Runtime Invariant Discovery rather than simple pattern matching.
Its objective is structural stability,
not textual similarity.

5. Discovery Rather Than Similarity

One of the most important consequences of this reinterpretation is the distinction between similarity and invariant discovery.
Similarity asks
"How alike are two observations?"
CCC asks
"What stable Runtime Invariant explains all observations?"
This distinction explains why CCC often succeeds even when conventional similarity measures fail.
The objective is structural identity,
not numerical proximity.

6. Relationship to Modern AI

Many successful AI systems perform operations closely related to Runtime Invariant Discovery.
Examples include

Transformer

- word embeddings,
- contextual embeddings,
- representation learning.

Vision AI

- feature extraction,

- multi-scale visual representations.

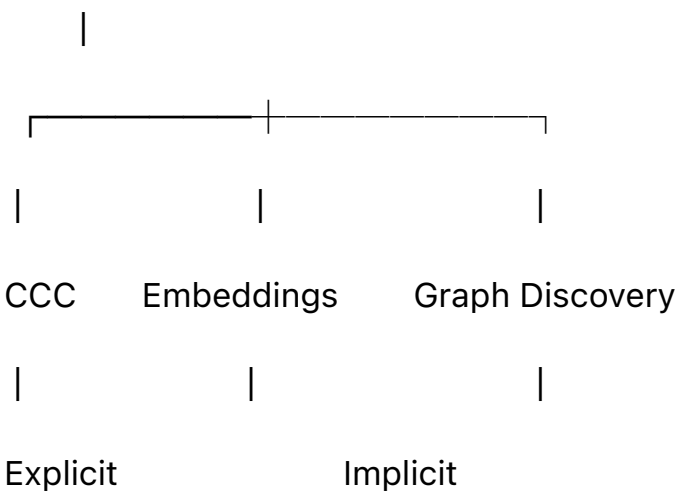
Graph AI

- graph embeddings,
- graph matching,
- structural pooling.

These methods frequently discover Runtime Invariants implicitly.
CCC performs the same Runtime Function explicitly.
Consequently,
CCC and modern representation learning should be viewed as complementary Runtime Invariant Discovery operators rather than competing paradigms.

7. CCC Within Runtime Invariant Discovery Algebra

Within Runtime Invariant Discovery Algebra,
CCC occupies a fundamental position.
Runtime Invariant Discovery



CCC therefore serves as one canonical Discovery Operator inside the broader Runtime Invariant Algebra.
Future Discovery Operators may differ in implementation,
while sharing the same structural objective.

8. Engineering Implications

Viewing CCC as Runtime Invariant Discovery immediately broadens its applicability.
CCC becomes applicable not only to structural reasoning,
but also to

- representation learning,

- runtime migration,
- model alignment,
- semantic compression,
- foundation models,
- collective learning,
- Runtime Invariant Engineering.

Rather than remaining an isolated algorithm,
CCC becomes a foundational Runtime Operator.

Key Takeaways

- CCC may be interpreted as an explicit Runtime Invariant Discovery Operator.
- The objective of CCC is discovering stable Runtime Invariants rather than measuring similarity.
- Runtime Invariant Discovery provides a higher-level interpretation of CCC.
- Modern AI often performs implicit Runtime Invariant Discovery, while CCC performs it explicitly.
- CCC naturally connects Structural Intelligence with Transformer-based AI, Graph AI, Vision AI, and future Runtime systems.
- Runtime Invariant Discovery Algebra provides a unified structural framework for interpreting these approaches.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-103 — Two-Way CCC as Bidirectional Runtime Invariant Discovery
- FTRIA-104 — XX Starmap as Multi-View Runtime Invariant Discovery
- FTRIA-105 — Differential Trees as Runtime Invariant Localization
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding
- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery

Foundations

- FTRIA-001 — Runtime Invariant Degrees of Freedom
- FTRIA-002 — RI-Preserving Transformation
- FTRIA-007 — Runtime-Invariant Configuration Space